

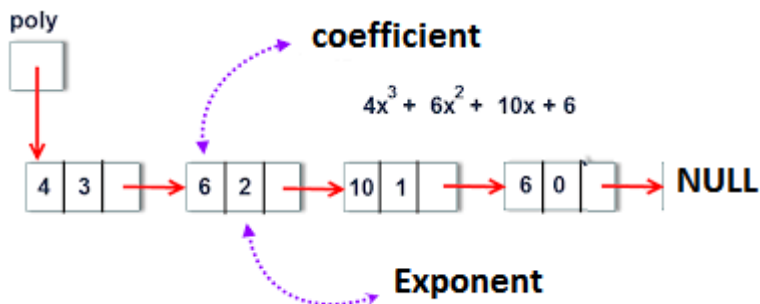
CSC310 Data Structures and Algorithm Analysis Programming Assignment 5

Write a program that ADDs and MULTIPLYs **two or more** (up to 10) polynomials in **one variable x** (that allows both positive and negative exponents).

The program would run until number of polynomials input is 0.

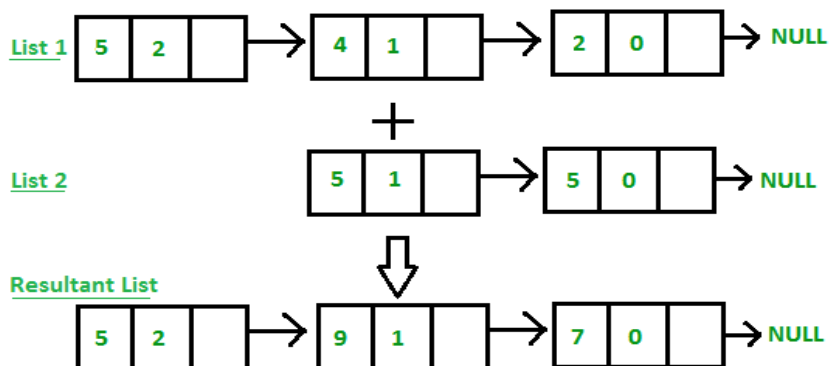
Consider the following linked list implementation declaration for **Polynomial** using pointers:

```
struct ATerm
{
    int      coefficient;
    int      exponent;
    ATerm*   next;
}
typedef ATerm* Polynomial
```



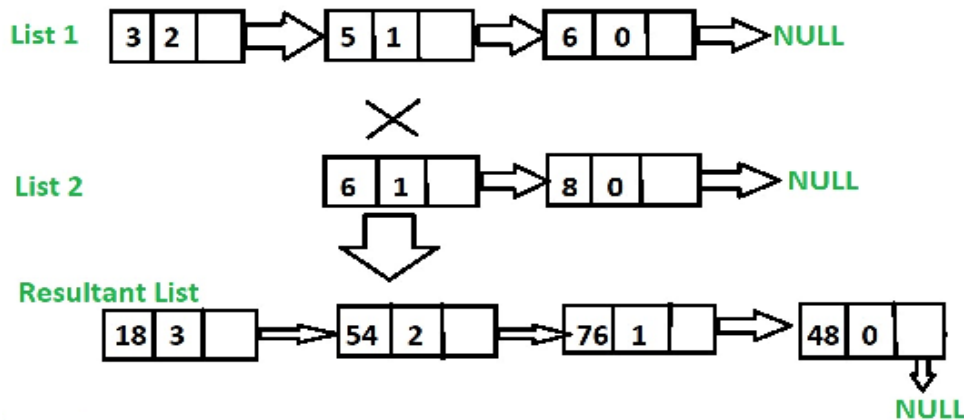
In your program, you should have at least two function prototypes such as:

Polynomial Add(const **Polynomial**, const **Polynomial**);
 // adds two polynomials and returns a polynomial which is the sum of these two polynomials.



Polynomial Multiply(const **Polynomial**, const **Polynomial**);

// multiplies two polynomials and returns a polynomial which is the product of these two polynomials.



To simplify the assignment, you are allowed to prompt the user for the number of polynomials to be input. A polynomial $5x^5+2x^4-3x^3+8x^2+9x+1$ can be input as $5\ x^5+2x^4-3x^3\ +\ 8\ x^2\ +\ 9x\ +\ 1$. **For user input, spaces between coefficients, operands, exponent and term are optional or more than one.**

An input term is defined as an optional coefficient integer c, character x, followed by optional exponent information specified as ^k, where k is an integer, i.e., cx^k

Note that any term to the power 1 is simply input as x (without the ^ information). A constant term K is entered as K. There won't be any term with zero coefficient (except the constant 0).

For the output, there is always **a space between each term of polynomial around the arithmetic operator “+” or “-” or “=” operators**. Each polynomial is enclosed by a pair of square brackets with no space before the first term and last term. There is no space between the coefficient, variable X, ^ and the exponent information. Note that the “=” operators are aligned vertically.

For simplicity, you can assume the following in the input format:

1. **Coefficients and exponents are within the range of int type integers.** Therefore, you don't need to how to handle values outside the int range. However, this means that **the value can be positive or negative**.
2. The polynomial is input in the format of **decreasing order of exponents**, with **like terms combined already** in the **simplest and most consolidated form**. **There will not be two terms of the same exponents**. The terms are connected via “+” or “-” operator only. Your result should be output in the same format in decreasing order of exponents, with like-terms combined and simplified.

For example, the input and output format look like:

```
Number of polynomials -> 2
Input polynomial 1 -> 5x+1
Input polynomial 2 -> 3 x^ 2 - 4 x
```

```
Sum = [5x + 1] + [3x^2 - 4x]
      = 3x^2 + x + 1
```

```
Product = [5x + 1] * [3x^2 - 4x]
          = 15x^3 - 17x^2 - 4x
```

```
Number of polynomials -> 3
Input polynomial 1 -> 7x ^2+ 1
Input polynomial 2 -> 3 x - 4
Input polynomial 3 -> -1 + x^ -2
```

```
Sum = [7x^2 + 1] + [3x - 4] + [-1 + x^-2]
      = 7x^2 + 3x - 4 + x^-2
```

```
Product = [7x^2 + 1] * [3x - 4] * [-1 + x^-2]
          = -21x^3 + 28x^2 + 18x - 24 + 3x^-1 - 4x^-2
```

```
Number of polynomials -> 0
```

You should use C++ for this assignment. You cannot call any library functions except those that deal with I/O and string handling functions.

Your program should execute interactively on your computer. Please test your program thoroughly and make sure it is free of errors. Points will be deducted for each error that I discover for you.

In addition, **your program should account for all possible user input errors.**

Please also pay attention to program design and follow all the good programming practice. I reserve the right to deduct points based on these criteria.

Submit: Program design
Hierarchical diagrams/ Structure charts
Procedure specification
Program listing
Testing cases with Justification
Results of test cases